Randomised Quickselect vs Deterministic Median of Medians

Advanced Algorithms Assessment 2

Team Members:

Jonathan Immanuel / 14487692

Colin Ice King-eo / 25650223

Table of Contents

1 The Problem and The Algorithms....3
1.1 Problem statement....3
1.2 Shared partitioning idea....3
1.3 Randomised Quickselect....3
1.4 Median of Medians....4
2 Theoretical Comparison and Predictions....4
2.1 Complexity and Implementation Costs....4
2.2 Predictions Before Performance Experiments....5
2.3 Regime Specific Expectations....6
3 Experimental Methodology....7
3.1 Research Questions and Correctness Verification....7
3.2 Shared Correctness Requirements....8
3.3 Benchmark Input Design....8
3.4 Build and Timing Procedure....9
3.5 Analysis and Reproducibility Controls....10
3.6 Reproduction Procedure....10
4 Results and Analysis....11
4.1 Overall Performance....11
4.2 Scaling and Ordering....12
4.3 Rank and Input Family Effects....13
4.4 Variation and Performance Review....13
5 What Surprised Us....14
5.1 Strongest Discrepancy....14
5.2 Revised Understanding of Selection Costs....14
5.3 Methodological Reflection on Test Coverage....15
6 What We Would Do Next....15
7 Individual Contributions....15
7.1 Jonathan....15
7.2 Colin....16
8 AI Use....16
8.1 Jonathan Immanuel....16
8.2 Colin Ice King-eo....16
References....18
Appendix....19

1 The Problem and The Algorithms

1.1 Problem statement

Our project compares Randomised Quickselect and Median of Medians for selecting a single order statistic. Given an unsorted integer array "arr" of length "n" and a zero-based rank "k" satisfying $0 \leq k < n$, the required result is b[k], where b is a copy of arr sorted into non-decreasing order. Duplicate occurrences occupy separate ranks. For example, arr = [9, 1, 4 ,7, 3] with k = 2 returns 4, while [2, 2, 5] with k = 1 returns 2.

Cerasa (2024) explains that selection supports quantiles and other statistical summaries without requiring a fully sorted array. Both repository functions accept an std::vector<int>& array and std::size_t rank, return an integer, and may rearrange the input while preserving its size and multiset. Empty input or $k \geq n$ raises std::out_of_range before mutation. The benchmark's middle rank is k = floor(n/2), selecting the upper middle value for an even n rather than averaging two middle values.

1.2 Shared partitioning idea

Both algorithms use partitionAroundPivot to divide the active interval [begin, end) into values smaller than, equal to, and greater than a pivot. With pivot 4, the example array could become [1, 3, 4, 9, 7]. Since two values precede the equal region, k = 2 returns 4 immediately. A rank outside that region directs the search into the appropriate strict side; neither side needs to be internally sorted.

The partition loop maintains four regions: smaller, equal, unclassified, and greater values. Each iteration classifies one previously unclassified element. The pivot is copied into a separate integer so swaps cannot alter it. A pass takes linear time in the active interval's length. Throughout selection, k remains an absolute vector index. Removing the non-empty equal region ensures progress and handles repeated pivot values together.

1.3 Randomised Quickselect

Randomised Quickselect selects a uniformly distributed index within the active interval using std::mt19937 and std::uniform_int_distribution. It partitions around that value, retains only the interval containing k, and repeats until k lies in an equal region. Its main search is iterative.

The convenience overload seeds a local generator through std::random_device. A second overload accepts an existing std::mt19937& without reseeding. The benchmark uses this second interface, preparing its generator before timing. Fixed seeds support reproducible pivot sequences within the same standard-library environment. Identical sequences across different libraries are not guaranteed.

Quickselect has expected $\Theta(n)$ running time over its random pivot choices on a fixed input. This does not require randomly-ordered input. Repeatedly poor pivots can retain almost the entire interval, producing the worst-case time of $\Theta(n^2)$ (Carnegie Mellon University, 2025). Randomness changes the work performed, while the returned answer remains exact. The iterative

implementation uses $O(1)$ auxiliary algorithm storage, including generator state whose size is independent of n.

1.4 Median of Medians

Median of Medians constructs its pivot systematically. It divides the active interval into groups of at most 5, sorts each group with std::sort, and swaps each group's middle value into a prefix of the same vector. It recursively selects the middle-ranked value of that prefix and uses this median of group medians as the partition pivot. For example, groups whose medians are 3, 6, and 9 produce pivot 6.

The pivot need not be the median of the active interval exactly. Its rank guarantees that either remaining strict side is bounded by a fixed fraction of the active interval, apart from rounding effects. Combined with recursive pivot selection, this gives $\Theta(n)$ worst-case running time (Carnegie Mellon University, 2025). Obtaining that guarantee requires group sorting and additional selection work.

For intervals containing at most five elements, the implementation sorts directly and returns values[k]. Main-interval narrowing is iterative; recursion occurs during pivot construction. Group medians remain inside the original vector, avoiding a separate median array, while recursive calls require $O(\log n)$ auxiliary stack space.

2 Theoretical Comparison and Predictions

2.1 Complexity and Implementation Costs

Both algorithms reduce selection to partitioning and searching one remaining interval. Their main theoretical difference concerns how reliably that interval shrinks. Table 1 summarizes their time and storage characteristics.

Characteristic	Randomised Quickselect	Median of Medians
Selection time	Expected Θ(n); worst-case Θ(n²)	Worst-case Θ(n)
Pivot Preparation	Random index generation and one pivot load	Sort groups of at most five, collect medians and recursively select a pivot
Auxiliary algorithm storage	O(1), including fixed-size generator state independent of (n)	O(log n) stack; medians stored within the input
Main search	Iterative	Iterative, with recursion during pivot construction
Duplicate handling	Three-way partition	Three-way partition

Characteristic	Randomised Quickselect	Median of Medians
Small intervals (< =5)	Still performs a randomised partition	Sorts directly and returns the requested value

Table 1: Theoretical characteristics of selected implementations

For Quickselect, selecting a pivot within the middle half of distinct ranks has probability of approximately one-half and leaves at most about three-quarters of the active values. The expected waiting time for this reduction is constant, producing linear expected scanning work. This expectation concerns random pivot choices on a fixed input. Carnegie Mellon University (2025) explains that repeated extreme pivots can instead retain (n - 1), (n - 2) and successively smaller intervals, producing quadratic work.

Median of medians guarantees at least $3n/10 - O(1)$ values on each non-strict side of its pivot. Consequently, either retained strict side contains at most $7n/10 + O(1)$ values. Writing T(n) for worst-case time and c for a constant gives the expression

$$T(n) \leq T(ceil(n/5)) + T(7n/10 + O(1)) + cn$$

The recursive fractions total 9/10, yielding a linear bound. Non-strict counting preserves the argument when values repeat (Carnegie Mellon University, 2025).

The implementations incur different constant costs. Quickselect draws random indices, whereas Median of Medians sorts small groups, moves their medians, and recursively processes that prefix. Both scan contiguous vector storage, but their comparisons, swaps, and memory accesses differ. Quickselect tests whether a value is smaller before testing whether it is greater, so minimum and maximum selections need not have identical costs. Sorted or nearly sorted input may also reduce the work performed by the small group sorts inside Median of Medians. These implementation details motivate performance hypotheses since they do not establish measured speedups.

Input order, requested rank, and repeated values can therefore affect practical performance without changing these general bounds. Three-way partitioning removes an entire equal region, while existing order can change swaps and branch behaviour. Quickselect's comparisons and swaps also fluctuate with its random choices, as examined by Ischebeck and Neininger (2024). Their probabilistic analysis motivates examining variation, rather than treating expected linear time as a fixed execution time. It does not establish percentiles of elapsed time for our duplicate-heavy inputs.

2.2 Predictions Before Performance Experiments

Before completing the performance experiment, we recorded six predictions about the expected results.

P1: Randomised Quickselect would be generally faster because randomised pivots require less work than recursive pivot selection

P2: Large all-equal arrays would be the input family with the largest difference in running time between the algorithms because Median of Medians must still construct its pivots whereas Quickselect has no need

P3: Sorted and reverse-sorted arrays will cause differences in measured time for Randomised Quickselect but are unlikely to cause quadratic behaviour

P4: We expected the requested rank k to produce a larger relative timing change for Median of Medians because it determines which partition remains after each step.

P5: Quickselect is more likely going to have variation in running time when its pivot seeds vary

P6: There is no defensible universal crossover size where Median of Medians could compete with Randomised Quickselect except for unusually slow Quickselect runs

2.3 Regime Specific Expectations

The following expectations are derived from the predictions outlined in section 2.2 and will be retained if measurements contradict them.

H1: Across random, nearly sorted and ordered inputs, Randomised Quickselect should usually have lower median selection time because its pivot preparation is cheaper. We will use its supplied generator interface to separate selection from generator construction.

H2: At matched large sizes and ranks, all equal arrays should produce the largest absolute timing gap, favouring Quickselect. Both algorithms still choose pivots and partitions but Quickselect avoids recursive median construction. Three-way partitioning should end each main search after one partition.

H3: Sorting or reversing the same values should change Quickselect's constants without leading to systematic quadratic growth. We will compare time per element across increasing sizes to distinguish ordering effects from scaling.

H4: Changing k between minimum, median, and maximum should produce a larger relative timing change for Median of Medians. Rank determines the retained partition in both algorithms, so this predicted difference requires testing rather than following directly from their designs. We will compare each algorithm's maximum-to-minimum median-time ratio across ranks.

H5: On identical inputs, varying pivot seeds should increase Quickselect's work and timing variation. We will compare within-input IQR and p95-to-median ratios, separating different-seed runs from the same seed repeats. Diagnostic work counts, where available, should distinguish pivot effects from timing noise. Median of Medians' deterministic work should remain constant, although elapsed time may fluctuate.

H6: Across n = 1,000 to 1,000,000, we expect no consistent crossover driven solely by size. Median of Medians may compete against unusually slow Quickselect runs despite losing on median time. We will compare matched inputs and report this as a tail-performance hypothesis.

These are predictions specifically. Ishebeck and Neininger's (2024) distinct-key analysis motivates studying fluctuations but supplies no wall-clock percentile guarantee for duplicate-heavy inputs.

3 Experimental Methodology

3.1 Research Questions and Correctness Verification

The experiment asks which implementation is faster across input families and increasing array sizes, and which family produces the largest timing gap. It examines how ordering, repeated values, and target rank affect performance, whether Median of Medians becomes competitive with unusually slow Quickselect runs, and how Quickselect's timing varies when its pivot seeds change. These questions operationalise P1-P6 without treating such predictions as findings. Selection time is the outcome of interest. Input generation, copying, generator initialisation and correctness checking must be excluded from the measured interval.

Verification used an unmodified temporary snapshot of the main branch in the repository (Immanuel & King-eo, 2026). GCC 13.3.0 compiled the five existing c++ test executables with the scripts' correctness flags: -std=c++23 -Wall -Wextra -Wpedantic -Werror. Both algorithm suites passed, with Quickselect reporting 137,246 selection checks. The benchmark input, case, and runner suites also passed. The PowerShell analysis verifier was inspected and executed. These outcomes establish successful checks for this revision, rather than proving correctness for every possible input.

The algorithm suites cover invalid inputs, singletons, hand-calculated examples, negative and repeated values, all-equal arrays and generated cases checked against std::sort. Their shared reference generator uses the seed 14487692, values from -20 to 20, four trials at each size from 1 to 80, and a subsequent 1,000-element shuffle. Every valid rank is checked in the small generated arrays. Reproducing the reference arrays requires matching generation calls and the standard-library environment.

Quickselect's suite additionally checks that selection preserves the vector's size and multiset while allowing its element order to change, repeated selections, and a supplementary matrix of six families over sizes from 1 to 10,000. The matrix exercises its convenience interface and four fixed pivot seeds, checking every rank for $n \leq 80$ and selected ranks thereafter. The Median of Medians suite does not contain this additional matrix. Consequently, the suites provide different workloads, and their total execution times cannot fairly compare algorithm speed.

The shared benchmark instead gives both algorithms identical input copies and checks each answer against a sorted reference outside timing. Its tests check input properties, seed reproducibility, valid case ranks, CSV structure, and alternating algorithm order. However, the runner changes Quickselect's pivot seed between repetitions without repeating each seed or recording operation counts. Its elapsed times therefore combine pivot effects with execution noise, and the current design cannot independently attribute the observed variation to either source.

3.2 Shared Correctness Requirements

Both APIs use zero-based k and must return the value at index k in a sorted copy of the original vector. Repeated occurrences occupy separate ranks. Valid calls may rearrange the vector while preserving its size and multiset. Invalid calls must throw std::out_of_range before mutation. Table 2 distinguishes these requirements from the available checks.

Requirement	Evaluation criterion	Current evidence
Empty or invalid input	Throw std::out_of_range.	Both suites test empty input and k = n.
One-element arrays	Return the sole element.	Both test [42] at k = 0.
Small examples	Match hand-calculated answers.	Both include aligned reference fixtures.
Sorted arrays	Select correctly from ascending input.	Quickselect fixtures; both algorithms benchmarked.
Reverse-sorted arrays	Select correctly from descending input.	Both have explicit descending fixtures.
Negative values	Respect signed integer ordering.	Both include mixed-sign fixtures and generated values.
Duplicates	Count repeated occurrences separately.	Both include named and generated cases.
All values equal	Return the repeated value.	Both include explicit all-equal fixtures.
Random arrays	Match a sorted-copy reference.	Both use the aligned reference generator.
Every valid k	Return the correct value at each rank.	Both enumerate ranks on generated sizes 1–80.
std::sort comparison	Return sorted_original[k].	Both generated suites; every benchmark result.

Table 2: Shared correctness requirements and current evidence

The benchmark checks selected values after timing, but does not assert size, multiset preservation or invalid-input non-mutation. Exhaustive rank checks concern the documented small correctness arrays, not every rank of the million-element benchmark.

3.3 Benchmark Input Design

The generator defines the six families in Table 3. The four distinct-value families share a multiset, allowing arrangement effects to be examined without changing the values. Separating random-unique from random-duplicates distinguishes disorder from repetition.

CSV family	Generation rule	Purpose
random-unique	Shuffle n consecutive integers starting at −floor(n/2).	Unordered distinct-key control.
random-duplicates	Uniform integers in [−3, 3].	Repeated keys; at most seven values.
sorted	The consecutive integers in ascending order.	Existing order.
reverse-sorted	Reverse the sorted sequence.	Opposite ordering.
nearly-sorted	Shuffle disjoint adjacent-pair indices; swap max(1, floor(n/100)) pairs.	Mild disorder: 1% swaps and 2% displaced positions at benchmark sizes.
all-equal	n copies of −7.	Extreme duplication and equal-region termination.

Table 3: Implemented benchmark families and their purposes

Full mode uses sizes 1,000, 2,000, 4,000, 8,000, 16,000, 32,000, 64,000, 128,000, 256,000, 500,000 and 1,000,000. Each family–size setting uses ranks 0, n/4, n/2, 3n/4 and n−1 with integer division. For these even sizes, n/2 is the upper middle order statistic, not an average.

Within each mode, numbering families and sizes from zero in their listed order, the input seed adds 100 times the family index and the size index to 14,487,692. The base pivot seed adds 100 times the family index and ten times the size index to 20,261,017. The five ranks share their input seed, reproducing identical contents with a matching standard library.

The full CSV contains 330 cases and 4,620 timing rows, with seven repetitions per algorithm. Each family–size setting samples one array; repetitions are not independent datasets. Smoke mode uses sizes 1,000 and 5,000 (60 cases). No timings below 1,000 elements are included.

3.4 Build and Timing Procedure

The recorded benchmark used MSYS2 g++ 16.1.0, as documented in results/benchmark_environment.txt. scripts/run_benchmarks.ps1 builds build/selection_benchmark.exe with -std=c++23 -O2 -DNDEBUG -Wall -Wextra -Wpedantic -Werror and the include directory. Both implementations are compiled into the same executable under identical flags. These benchmark settings differ from the independent correctness build in Section 3.1.

selection_benchmark::writeBenchmarkResults performs one untimed warm-up per algorithm for each case, followed by seven repetitions numbered 0-6. Median of Medians runs first on even repetitions and Quickselect first on odd repetitions. This alteration reduces consistent ordering bias without randomising the case schedule.

Std::chrono::steady_clock brackets only selection::deterministicSelect or the supplied-generator overload of selection::randomisedSelect. Input generation, reference sorting, vector copying, std::mt19937 construction, checking, and CSV writing occur outside timing. Random pivot

draws remain inside the measured selection work. Quickselect receives the case's base pivot seed plus the repetition number. Its convenience interface and entropy acquisition are not benchmarked. Returned values remain observable through checking and CSV output. Durations are stored in nanoseconds, but the implementation does not calibrate timer overhead or establish nanosecond accuracy.

3.5 Analysis and Reproducibility Controls

scripts/summarise_benchmarks.ps1 groups observations by family, n, k, and input seed, then by algorithm. It requires repetitions 0-6 and agreement among returned values. Each seven-run group produces MedianNs, MinNs, MaxNs, and SpreadPercent, defined as 100 times the maximum-minus-minimum range divided by the median. No observations are discarded (Immanuel & King-eo, 2026).

QuickselectSpeedup is the Median of Medians case median divided by Quickselect's case median, rounded to three decimal places where values above one favour Quickselect. The size summary takes the median of the five rank medians. Accordingly, median_time_by_size.svg shows that aggregate, rank_effect_at_largest_size.svg retains separate rank medians, and speedup_at_largest_size.svg shows the median of five case speedup ratios at the largest size.

scripts/verify_benchmark_analysis.ps1 independently recalculates archived case medians and speedup ratios, checks summary counts, and validates non-empty SVG documents. Raw input_seed and pivot_seed fields preserve the generation table. However, the analysis does not provide IQR, confidence intervals, or p95 estimates. Seven changing-seed measurements on one input per family and size support descriptive comparisons. They cannot isolate pivot randomness from execution noise or establish reliable tail probabilities. The maximum observed time is not a worst-case complexity bound.

3.6 Reproduction Procedure

From a separate checkout of repository commit 4140782, run the following commands at the repository root in PowerShell, with g++ available through PATH or the documented MSYS2 fallback. The reproduced directory keeps new measurements separate from the archived results.

```
powershell -ExecutionPolicy Bypass -File scripts/verify.ps1

powershell -ExecutionPolicy Bypass -File scripts/run_benchmarks.ps1 -Mode smoke -OutputPath results/reproduced/smoke_benchmark.csv

powershell -ExecutionPolicy Bypass -File scripts/run_benchmarks.ps1 -Mode full -OutputPath results/reproduced/full_benchmark.csv

powershell -ExecutionPolicy Bypass -File scripts/summarise_benchmarks.ps1 -InputPath results/reproduced/full_benchmark.csv -OutputDirectory results/reproduced
```

The analysis creates benchmark_case_summary.csv (660 rows), benchmark_size_summary.csv (132 rows), benchmark_speedup_summary.csv (330 rows), and three SVGs named in Section 3.5. Verify.ps1 validates the archived CSV through its analysis verifier, writing check outputs under build/benchmark_analysis_check. Retain results/benchmark_environment.txt as the

original-run record and record the new environment separately. Dataset definitions and summary calculations should reproduce, but elapsed times may differ.

4 Results and Analysis

4.1 Overall Performance

After performing the benchmark tests, Randomised Quickselect had the lower median in 324 of 330 matched cases (98.2%), supporting P1 (Immanuel & King-eo, 2026). Figure 1 compares central-rank performance across all six families, while Table 4 provides representative cases. For one million shuffled distinct values, Quickselect took 10.579 ms against 33.831 ms for Median of Medians, a 3.198-fold speedup.

Avoiding group sorting and recursive pivot construction plausibly explains this advantage, although those costs were not measured separately. Median of Medians nevertheless won six case medians, all at n < 4,000. Each median summarises seven calls on one input, so those observations describe the recorded benchmark rather than performance across independently sampled datasets.

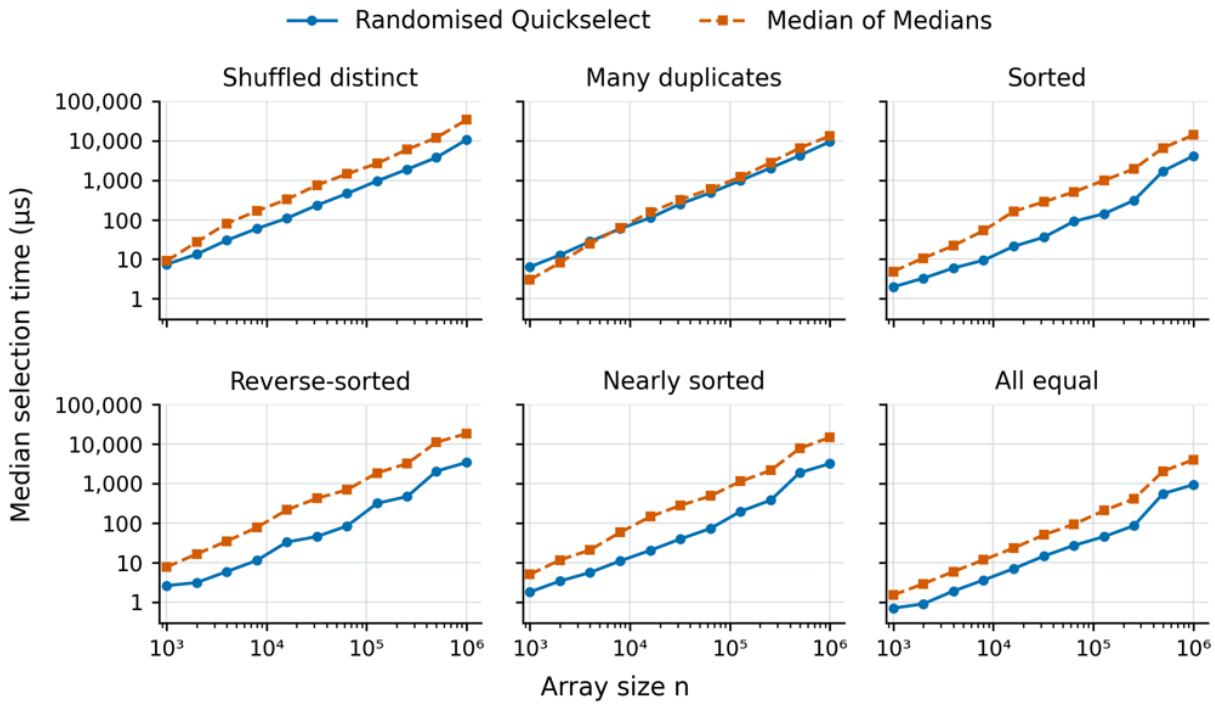


Figure 1: Median selection time across six input families at k = n/2

Input family	n	k	MoM (µs)	QS (µs)	S
Shuffled distinct	1,000,000	500,000	33,830.5	10,578.7	3.198

Many duplicates	1,000	500	3.0	6.4	0.469

Table 4: Selected central-rank case medians

Notes: Figure 1 uses seven-call medians and shared logarithmic axes. For Table 4, MoM = Median of Medians, QS = Randomised Quickselect, S = MoM/Qs. All seven oracle checks passed per algorithm and case.

4.2 Scaling and Ordering

Figure 2a shows fluctuating time per element on sorted and reverse-sorted inputs across n = 1,000-1,000,000 (Immanuel & King-eo, 2026). At n = 1,000,000, Quickselect took 4.092ns/element on sorted input and 3.418ns/element on reverse-sorted input. These curves are compatible with linear scaling and P3, without establishing an asymptotic bound. Reordering values can affect partition comparisons, swaps, and the work of Median of Medians' group sorts. However, family-specific pivot seeds prevent attributing Quickselect's timing differences solely to ordering.

Figure 2b shows Median of Medians winning the duplicate-heavy central-rank medians at n = 1,000, 2,000, and 4,000. Quickselect wins from 8,000 onwards. Using S = Median of Medians median / Quickselect median, the ratio rises from 0.876 at 4,000 to 1.064 at 8,000. This qualifies P6 by identifying a candidate crossover for one regime. Denser sizes and repeated sessions are still needed before claiming a stable threshold.

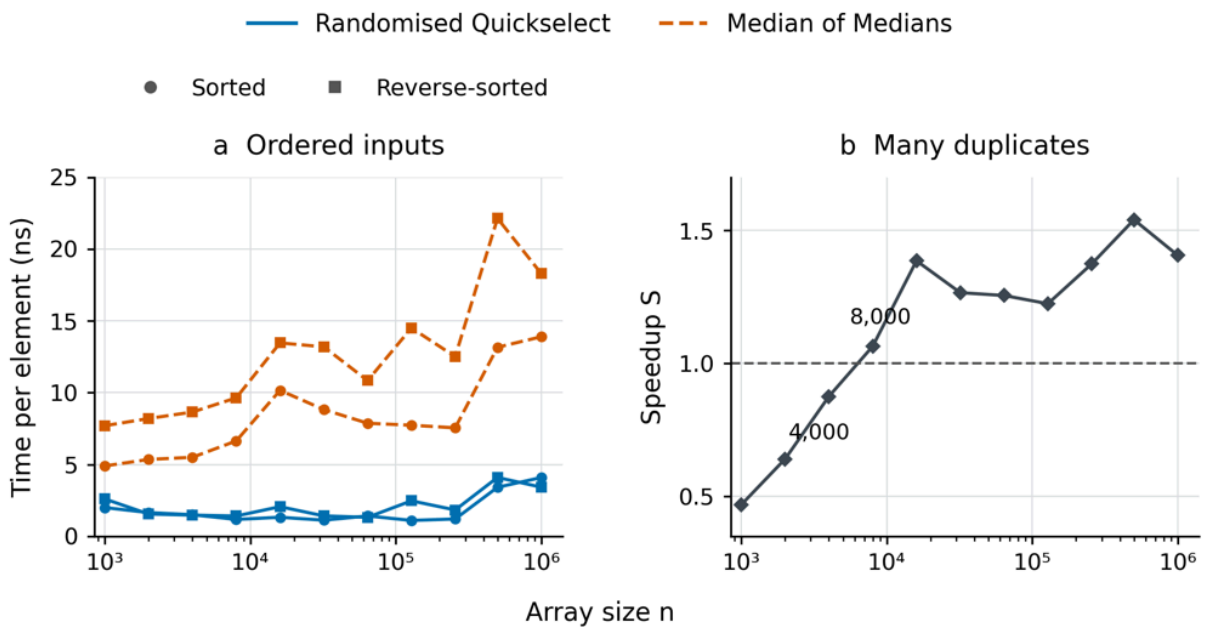


Figure 2: Central-rank scaling and duplicate-family crossover

Notes: k = n/2 for seven-call medians. Horizontal axes are logarithmic. S < 1 means MoM favour while S > 1 means Quickselect favour.

4.3 Rank and Input Family Effects

At n = 1,000,000, the median paired timing gap (Median of Medians minus Quickselect) across five ranks was 2.844 ms for all-equal inputs (Table 5), contradicting P2 (Immanuel & King-eo, 2026). Both algorithms resolve the main all-equal search after one three-way partition, although Median of Medians still constructs its pivot. Shuffled distinct inputs can require additional partitions and more group-sorting work, but those operation counts are not recorded.

Rank sensitivity also generally contradicted P4. Let R be the maximum/minimum case median across k = 0, n/2 and n − 1. Quickselect had the larger R in 56 of 66 family-size settings. For million-element sorted inputs, R was 3.353 for Quickselect and 1.185 for Median of Medians. The requested rank determines which strict side remains, while a large equal region can end the search immediately. The all-equal R values in Table 5 cannot demonstrate extra search partitions, so execution effects remain a possible explanation.

Family	Gap (ms)	R (MoM)	R (QS)
Shuffled distinct	26.574	1.252	2.219
Many duplicates	10.862	1.558	1.455
Sorted	12.494	1.185	3.353
Reverse-sorted	17.120	1.306	1.849
Nearly sorted	13.915	1.213	1.910
All equal	2.844	2.466	2.357

Table 5: Family Gaps and Rank Sensitivity

Notes: n = 1,000,000, MoM = Median of Medians, QS = Randomised Quickselect

4.4 Variation and Performance Review

Quickselect's relative interquartile range (IQR divided by median) was larger in 306 of 330 cases. Across such cases, its median relative IQR was 30.99%, compared with 7.77% for Median of Medians, consistent with P5 (Immanuel & King-eo, 2026). Post-processing used linearly interpolated quartiles for each algorithm's seven timings.

Quickselect's changing seeds and execution noise cannot be separated without same-seed repetitions or work counts. Seven observations cannot establish reliable p95 estimates. Median of Medians' median was below Quickselect's observed maximum in 16 cases, including ten where Quickselect won on median time. These maxima provide limited evidence about unusually slow runs. Table 6 reviews all six predictions.

Prediction	Assessment
P1 / H1	Broadly supported
P2 / H2	Contradicted
P3 / H3	Compatible with measurements
P4 / H4	Contradicted overall
P5 / H5	Descriptively consistent
P6 / H6	Mixed

Table 6: Review of Recorded Predictions

5 What Surprised Us

5.1 Strongest Discrepancy

The clearest discrepancy concerned P2/H2. We expected large all-equal arrays to produce the largest absolute timing gap because Quickselect avoids recursive pivot construction. At n = 1,000,000, Table 5 instead shows a median paired gap (Median of Medians minus Quickselect) across five ranks of 2.844 ms for all-equal inputs and 26.574ms for shuffled distinct inputs (Immanuel & King-eo, 2026). These were respectively the smallest and largest gaps among the six families. Quickselect still selects a random pivot, but our emphasis on pivot preparation failed to predict the family ordering.

5.2 Revised Understanding of Selection Costs

Selection costs depend on how much work the process of partitioning eliminates and the pivot work required along the remaining search. On all-equal benchmark inputs, both main searches finish after one full partition. Median of Medians still sorts groups and recursively selects from an all-equal median prefix through choosePivot. Shuffled distinct inputs can require additional narrowing and group processing, plausibly enlarging its extra work (Immanuel & King-eo, 2026).

Table 8 (Representative central-rank cases at n = 1,000,000) in the Appendix gives Quickselect median speedups of 4.283 times on all-equal inputs and 3.198 times on shuffled distinct inputs at k = n/2. The corresponding absolute gaps are 3.045ms and 23.252ms. A larger proportional advantage therefore accompanies a smaller absolute saving. These central-rank gaps differ from Table 5's median paired gaps across five ranks.

Table 5 also reveals an all-equal anomaly: maximum/minimum median-time ratios across k = 0, n/2, and n-1 are 2.466 for Median of Medians and 2.357 for Quickselect. For every valid rank, both main searches terminate in the first equal region. These timing changes do not demonstrate

extra search partitions. Execution effects and the fixed case schedule are possible explanations whose contributions remain unmeasured.

These findings revise our prior understanding. Early duplicate elimination can reduce work across both the main search and recursive pivot construction. Deterministic complexity guarantees nevertheless leave an elapsed time variable. The implementation and timings are consistent with this account, but partition counters, group sort timings, and same-seed Quickselect repetitions were not recorded, so specific contributions cannot be isolated.

5.3 Methodological Reflection on Test Coverage

Quickselect's standalone suite includes testSupplementaryMatrix, input-preservation assumptions, and repeated-selection checks absent from the Median of Medians suite (Immanuel & King-eo, 2026). Passing both suites does not establish equal coverage, and their total execution times cannot fairly compare algorithm speed. As Section 3.1 explains, the shared benchmark times selection on identical input copies. Our performance conclusions therefore rely on matched benchmark cases, with the coverage difference reported separately.

6 What We Would Do Next

Koskela et al. (2023) suggest benchmarking independently generated inputs across sessions, randomising case order, and retaining environment records to improve reproducibility. Repeating fixed pivot seeds alongside varying seeds will help distinguish execution effects from pivot randomness. Separate diagnostic runs will count comparisons, swaps, scanned elements, and equal-region sizes, motivated by Quickselect cost fluctuations (Ischebeck & Neininger, 2024). Timing runs will omit these counters. To vary duplicate cardinality and rank and broaden instance coverage as recommended by Smith-Miles and Muñoz (2023), more sample sizes between 4,000 and 8,000 would be sampled to more closely identify the candidate crossover. C++ adaptations of simpleselect, implemented as quickselectFS (Azzini et al., 2023), and OSILA (Cerasa, 2024) can be compared against both baselines under the same zero-based contract and correctness checks. Required shuffling and sampling could be timed while input copying and generator construction will remain excluded. Larger samples will be required in order to perform the estimation of tail percentiles.

7 Individual Contributions

7.1 Jonathan

- Implemented median of medians
- Created the github repository and ran the necessary programs/powershell scripts
- Contributed to the shared benchmark generator, timing procedure, repeated-run checks, CSV output, and PowerShell analysis scripts
- Set the schedule for the planning and implementation of the repository and creation of the report
- Created the presentation slides for video

7.2 Colin

- Implemented randomised quicksort
- Planned out research report and contents with AI assistance
- Typed out main draft and final contents of the research report for sections 1-6
- Performed analysis of benchmarking results in main repository for use in the report

8 AI Use

8.1 Jonathan Immanuel

I used Codex while developing and testing the Median of Medians implementation. It helped me plan the files, work through the group-of-five pivot method, suggest edge cases and prepare benchmark scripts.

One incorrect suggestion described an experiment with five warm-ups, three separate sessions, one hundred Quickselect seeds, operation counters, and a different optimisation level. That was not the experiment we ran. I rewrote the methodology to match the real benchmark: one warm-up, seven timed repetitions, fixed input seeds, changing Quickselect pivot seeds, and compilation with -O2.

A second mistake occurred in the benchmark validator. The suggested code expected repetition numbers 1 through 7, while the C++ runner recorded repetitions 0 through 6. This produced hundreds of false missing-repetition errors. I compared the script with the raw CSV and corrected the expected numbering.

A third problem occurred in the PowerShell SVG generation. An incorrectly formatted string prevented one plot from being created properly. I checked the generated file, corrected the formatting, and regenerated the figures.

Another incorrect conclusion was that Quickselect was faster in every benchmark case. The case summary showed that it was faster in 324 of 330 cases, while Median of Medians won six cases. I corrected the report and retained the six exceptions in the results.

I understand the selection contract, three-way partitioning, group-of-five construction, the $3n/10$ and $7n/10$ bounds, and how the benchmark kept setup work outside the timer. I relied more on AI for PowerShell and SVG syntax because I had less experience with those tools, but I checked the resulting files and calculations before using them.

8.2 Colin Ice King-eo

ChatGPT and Codex are used in both the planning of the implementation of randomised quickselect and the contents of the report. They helped organise the assignment into development tasks, research

questions, predictions, and a report structure. Codex in particular is used to develop the Randomised Quickselect algorithm with three-way partitioning, zero-based k, invalid-input exceptions, a supplied generator interface, and specific tests against std::sort.

The plot plan proposed by the AI features tiny-input latency across both algorithms' interfaces. This was not followed because the final version of the benchmark algorithm starts at an input size n of 1,000 and uses only the supplied-generator interface. We decided to revise the output plan and replaced the original plot with recorded rank profiles, leaving the crossover testing for Section 6. The proposal was a possible experiment, but it could not support a results figure from the available data.

The AI also proposed a variation analysis, which requires a different experiment from the one we performed. The plan included same-seed comparisons, diagnostic work counts, and p95-based analysis. We instead limited Section 4.4 to descriptive spread, calculated IQR explicitly through post-processing, and identified observed maxima as sample maxima. We also retained the limitation that pivot effects and execution noise.

A strange mistake the AI made is misclassifying a research algorithm. The original AI plan grouped Cerasa's OSILA with extensions that changed the selection task. Cerasa (2024) describes an algorithm that returns the exact requested order-statistic. We rechecked the paper manually and revised section 6 to propose OSILA more closely to its actual description of being an exact-selection comparator.

AI helped create a common technical language across two algorithm branches. Concrete criteria for reviewing contributions are also made easily thanks to their inclusion in the project. The benchmark runner also gave us a common timing boundary, avoiding comparisons between standalone test workloads.

The main risk was accepting a fluent explanation of the repository without tracing it to my partner's code and measurements. This caused the mismatch between the readme's claim of quickselect beating out median of medians in all tests and the actual results. The report fixes this by retaining the exceptions. Our experience has nonetheless supported using AI outputs as material for shared review. Understanding each algorithm, checking integration, and defending the statistical interpretation remain group responsibilities. Keeping the six predictions visible also preserved the distinction between a reasonable expectation contradicted by evidence and an incorrect factual claim which the AI could've made.

References

Azzini, I., Perrotta, D., & Torti, F. (2023). *A practically efficient fixed-pivot selection algorithm and its extensible MATLAB suite* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2302.05705

Carnegie Mellon University. (2025). *Introduction and linear-time selection* [Lecture notes, 15-451 Algorithm Design and Analysis]. https://www.cs.cmu.edu/~15451-s25/notes/lecture01.pdf

Cerasa, A. (2024). Order statistics in large arrays (OSILA): A simple randomised algorithm for a fast and efficient attainment of the order statistics in very large arrays. *Computational Statistics, 39*, 3599–3624. https://doi.org/10.1007/s00180-023-01381-1

Ischebeck, J., & Neininger, R. (2024). On fluctuations of complexity measures for the FIND algorithm. In C. Mailler & S. Wild (Eds.), *35th International Conference on Probabilistic, Combinatorial and Asymptotic Methods for the Analysis of Algorithms* (Leibniz International Proceedings in Informatics, Vol. 302, Article 9, pp. 9:1–9:15). Schloss Dagstuhl – Leibniz-Zentrum für Informatik. https://doi.org/10.4230/LIPIcs.AofA.2024.9

Immanuel, J., & King-eo, C. I. (2026). *Deterministic and randomized selection* (Commit 4140782) [Computer software]. GitHub. https://github.com/jo-nuel/Advanced-algorithm-Ass-2/tree/41407823025dba542999803ea0e425ea348d1352/results

Koskela, T., Christidi, I., Giordano, M., Dubrovska, E., Quinn, J., Maynard, C., Case, D., Olgu, K., & Deakin, T. (2023). Principles for automated and reproducible benchmarking. In

Proceedings of the SC '23 workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis (pp. 609–618). Association for Computing Machinery. https://doi.org/10.1145/3624062.3624133

Serang, O. (2023). *Median of heaps: Linear-time selection by recursively constructing binary heaps* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2304.12313

Smith-Miles, K., & Muñoz, M. A. (2023). Instance space analysis for algorithm testing: Methodology and software tools. *ACM Computing Surveys, 55*(12), Article 255. https://doi.org/10.1145/3572895

Appendix

Input family	Algorithm	Median (µs)	Mean (µs)	IQR (µs)	S	Oracle checks
Shuffled distinct	Median of Medians	9.0	9.6	3.05	1.216	7/7
Shuffled distinct	Quickselect	7.4	7.7	1.10		7/7
Many duplicates	Median of Medians	3.0	5.9	6.05	0.469	7/7
Many duplicates	Quickselect	6.4	6.8	2.40		7/7
Sorted	Median of Medians	4.9	5.1	0.25	2.450	7/7
Sorted	Quickselect	2.0	2.0	0.55		7/7
Reverse-sorted	Median of Medians	7.7	7.8	0.40	2.962	7/7
Reverse-sorted	Quickselect	2.6	2.5	0.25		7/7
Nearly sorted	Median of Medians	5.0	5.1	0.15	2.778	7/7
Nearly sorted	Quickselect	1.8	1.8	0.10		7/7
All equal	Median of Medians	1.5	1.5	0.05	2.143	7/7
All equal	Quickselect	0.7	0.7	0.05		7/7

Table 7: Representative central-rank cases at n = 1,000

Notes: k = n/2 (zero-based upper-middle rank). Each row contains seven calls. Quickselect denotes Randomised Quickselect using the supplied-generator interface. Mean and Type 7 IQR are new calculations from raw observations. S is the paired median ratio (Median of Medians / Quickselect), shown once per input; S > 1 favours Quickselect. Oracle status refers to recorded successful checks in the benchmark runner.

Input family	Algorithm	Median (µs)	Mean (µs)	IQR (µs)	S	Oracle checks
Shuffled distinct	Median of Medians	33,830.5	33,879.7	1,192.90	3.198	7/7
Shuffled distinct	Quickselect	10,578.7	11,669.2	2,258.45		7/7
Many duplicates	Median of Medians	13,225.9	13,881.3	2,181.10	1.406	7/7
Many duplicates	Quickselect	9,405.9	9,587.0	2,253.55		7/7
Sorted	Median of Medians	13,891.3	15,265.4	3,622.30	3.395	7/7
Sorted	Quickselect	4,091.6	4,277.1	1,807.00		7/7
Reverse-sorted	Median of Medians	18,287.0	18,461.8	808.60	5.350	7/7
Reverse-sorted	Quickselect	3,418.2	3,377.2	873.30		7/7
Nearly sorted	Median of Medians	14,711.3	14,706.2	535.20	4.621	7/7
Nearly sorted	Quickselect	3,183.4	3,077.9	1,055.30		7/7
All equal	Median of Medians	3,972.9	3,898.9	237.85	4.283	7/7
All equal	Quickselect	927.6	935.4	76.65		7/7

Table 8: Representative central-rank cases at n = 1,000,000

Notes: k = n/2 (zero-based upper-middle rank). Each row contains seven calls. Quickselect denotes Randomised Quickselect using the supplied-generator interface. Mean and Type 7 IQR are new calculations from raw observations. S is the paired median ratio (Median of Medians / Quickselect), shown once per input; S > 1 favours Quickselect. Oracle status refers to recorded successful checks in the benchmark runner.

Input family	MoM (ms)	QS (ms)	Gap (ms)	Median S	MoM rank ratio	QS rank ratio
Shuffled distinct	36.906	10.550	26.574	3.419	1.252	2.219
Many duplicates	19.137	8.155	10.862	2.313	1.558	1.455
Sorted	15.683	3.566	12.494	4.238	1.185	3.353
Reverse-sorted	20.009	2.889	17.120	6.925	1.306	1.849
Nearly sorted	15.886	2.780	13.915	6.209	1.213	1.910
All equal	3.857	0.936	2.844	3.810	2.466	2.357

Table 9: Family differences at n = 1,000,000

Notes: MoM = Median of Medians; QS = Randomised Quickselect. Times are medians of five rank medians. Gap is the median of five paired differences; Median S is the median of five case ratios. Rank ratio is the maximum/minimum median time across k = 0, n/2 and n−1 only. The gap and speedup aggregates are calculated separately from the time aggregates.

Input family	n	k	MoM median (µs)	QS median (µs)	S
Many duplicates	1,000	500	3.0	6.4	0.469
Many duplicates	1,000	750	5.3	6.6	0.803
Many duplicates	1,000	999	4.4	5.0	0.880
Many duplicates	2,000	1,000	8.2	12.8	0.641
Many duplicates	4,000	2,000	24.7	28.2	0.876
Shuffled distinct	1,000	750	5.7	6.9	0.826

Table 10: All six Median of Medians case-median wins

Notes: Seven calls per algorithm and case; k is zero-based. S < 1 favours Median of Medians. For central-rank many-duplicate inputs, MoM wins at n = 1,000, 2,000 and 4,000; Quickselect wins from n = 8,000 onwards. The interval (4,000, 8,000) is a candidate regime-specific crossover requiring denser sizes and repeated sessions. It is not a universal threshold.

Prediction	Assessment	Observed evidence and limits
P1 / H1	Broadly supported	Quickselect has the lower case median in 324/330 cases (98.2%). Six exceptions remain. Pivot construction costs were not measured separately.

P2 / H2	Contradicted	At n = 1,000,000, the median paired gap across ranks is 2.844 ms for all-equal and 26.574 ms for shuffled distinct. All-equal is not the largest-gap family.
P3 / H3	Compatible with measurements	Sorted and reverse-sorted time-per-element curves differ. They show no systematic quadratic pattern over these sizes; finite timings do not prove an asymptotic bound.
P4 / H4	Contradicted overall	Using only k = 0, n/2 and n−1, MoM has the larger max/min median-time ratio in 10/66 family–size settings; Quickselect in 56/66.
P5 / H5	Descriptively consistent	Quickselect has higher relative IQR in 306/330 cases. Median case relative IQR is 30.99% for QS and 7.77% for MoM. Pivot causality and p95 remain untested.
P6 / H6	Mixed	MoM wins six case medians. The central-rank duplicate family changes winner between n = 4,000 and 8,000; this is a candidate regime-specific crossover, not a universal threshold. MoM's median is below the observed QS maximum in 16/330 cases (10 additional cases where QS wins the median); no tail guarantee follows.

Table 11: Evidence for the Assessments of the six Predictions

Notes: Results describe one input per family and size. Relative IQR is IQR divided by the case median. No work counters, same-seed Quickselect repeats or reliable tail-percentile estimates are available.